



Guia de Eventos em JavaScript

Da interatividade ao Objeto Event — um tutorial passo a passo para dominar como o JavaScript captura e responde a ações do usuário no navegador.

[</> JAVASCRIPT](#)[⚡ EVENTOS](#)[📖 TUTORIAL](#)

Agenda

O que vamos aprender?

01

Introdução aos Eventos

O papel do DOM e a sintaxe do
`addEventListener`

02

Mouse, Teclado e Formulário

Eventos essenciais para interações cotidianas

03

Ciclo de Vida da Página

`DOMContentLoaded`, `resize` e `scroll`

04

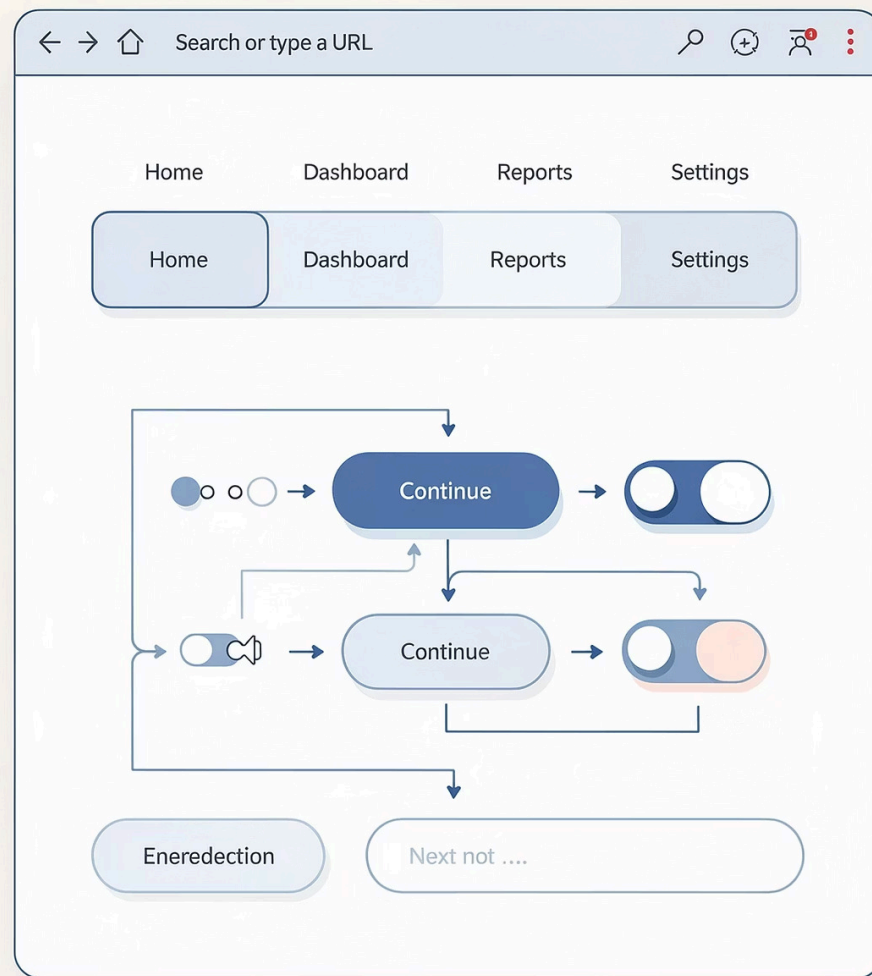
Deep Dive: Objeto Event

`event.target` e `preventDefault`

CAPÍTULO 1

Introdução aos Eventos

Eventos são sinais disparados pelo navegador sempre que algo acontece na página — um clique, uma tecla pressionada, um formulário enviado. O JavaScript "escuta" esses sinais e executa funções em resposta, tornando as páginas vivas e interativas.



O Papel do DOM e o addEventListener

O que é o DOM?

O **Document Object Model** é a representação em árvore de todos os elementos HTML. O JavaScript acessa e manipula esses elementos através do DOM para reagir a interações do usuário em tempo real.

- Cada tag HTML vira um **nó** na árvore
- Eventos são registrados em nós específicos
- O navegador gerencia a fila de eventos automaticamente

Sintaxe do addEventListener

A forma moderna e recomendada de registrar eventos:

```
// Seleciona o elemento
const botao = document.querySelector('#botao');

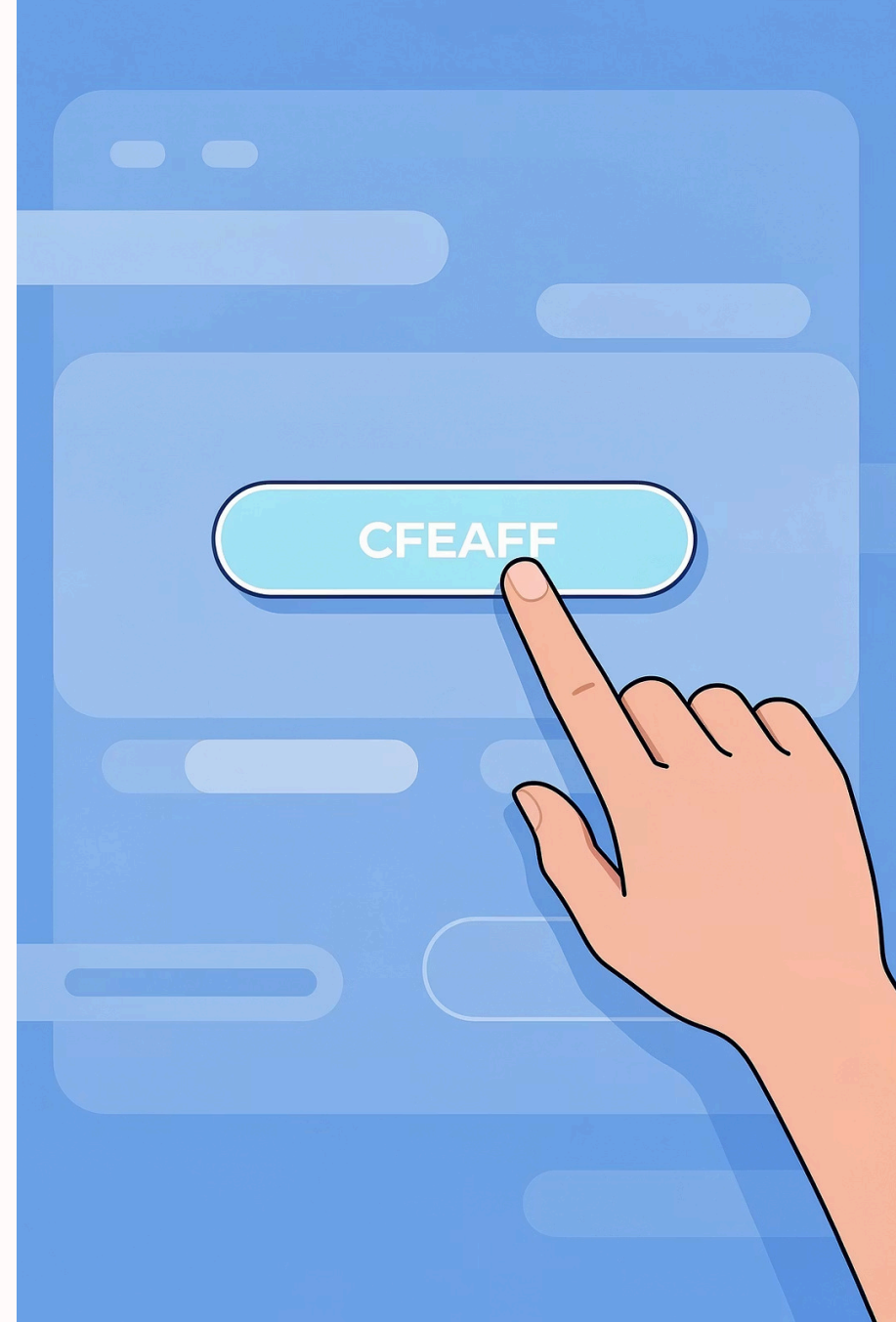
// Registra o evento
botao.addEventListener('click', function(event) {
  console.log('Botão clicado!', event);
});

// Com arrow function (ES6+)
botao.addEventListener('click', (e) => {
  console.log('Elemento:', e.target);
});
```

CAPÍTULO 2

Eventos de Mouse

Os eventos de mouse são os mais comuns em interfaces web. Eles permitem que você responda a cliques simples, duplos cliques e ao movimento do cursor sobre elementos — fundamentais para botões, menus e efeitos de hover.



Os 4 Principais Eventos de Mouse



click

Disparado ao clicar e soltar o botão do mouse sobre um elemento. Uso mais comum: botões de ação, links e toggles.

```
btn.addEventListener('click', () => {  
  alert('Clicado!');  
});
```



dblclick

Disparado com dois cliques rápidos. Útil para edição inline ou ações especiais que precisam de confirmação dupla.

```
elem.addEventListener('dblclick', () => {  
  elem.contentEditable = true;  
});
```



mouseover

Acionado quando o cursor entra na área do elemento. Ideal para efeitos de hover em menus e cards interativos.

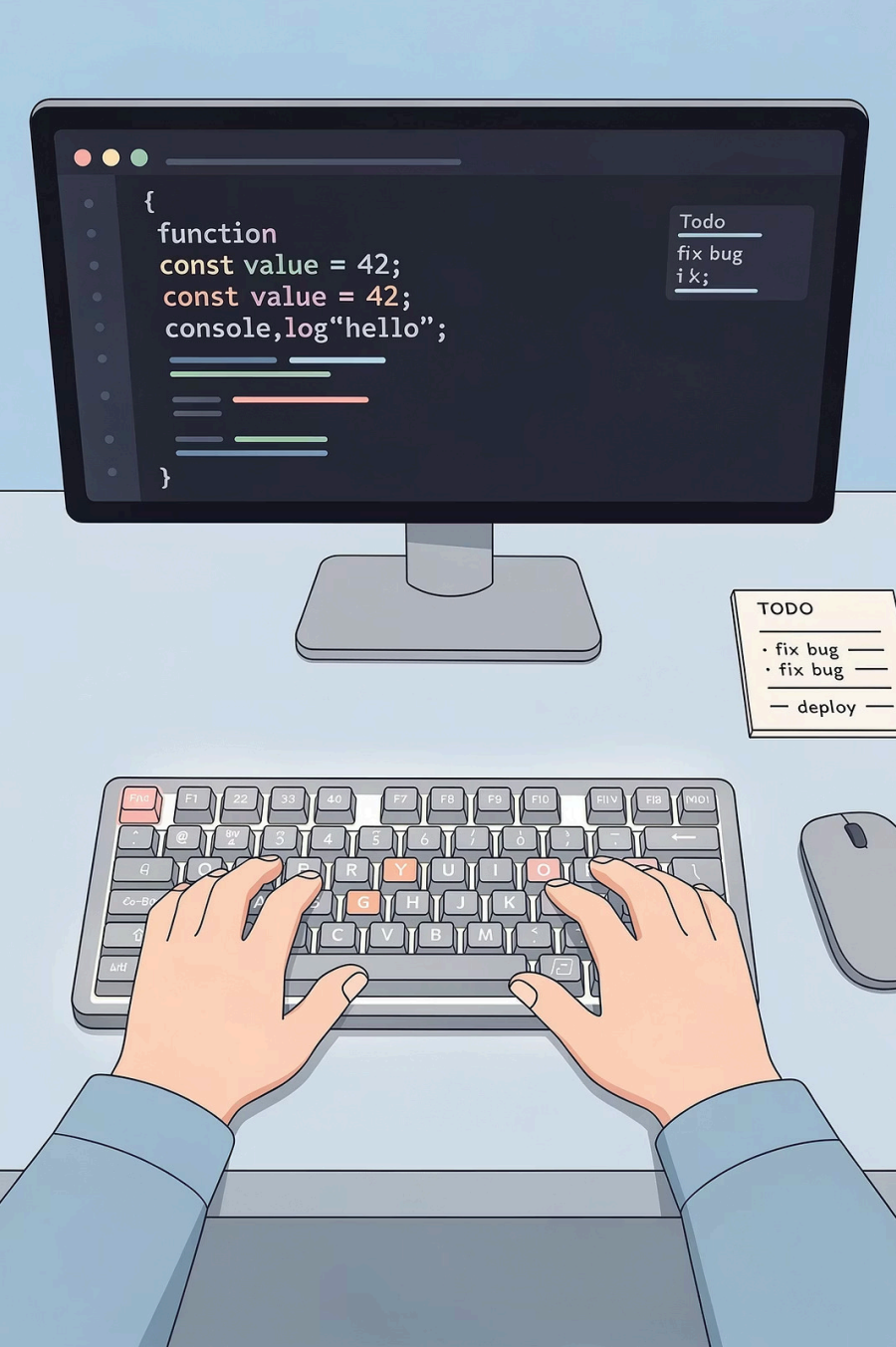
```
menu.addEventListener('mouseover', () => {  
  menu.classList.add('ativo');  
});
```



mouseout

Acionado quando o cursor sai da área do elemento. Usado em conjunto com `mouseover` para criar efeitos completos.

```
menu.addEventListener('mouseout', () => {  
  menu.classList.remove('ativo');  
});
```



CAPÍTULO 3

Eventos de Teclado

Eventos de teclado permitem capturar cada tecla pressionada pelo usuário — essenciais para atalhos de teclado, validação em tempo real, jogos e navegação acessível.

keydown vs keyup

keydown – Tecla Pressionada

Disparado **imediatamente** quando o usuário pressiona uma tecla. Repete enquanto a tecla fica pressionada. Ideal para atalhos e ações contínuas.

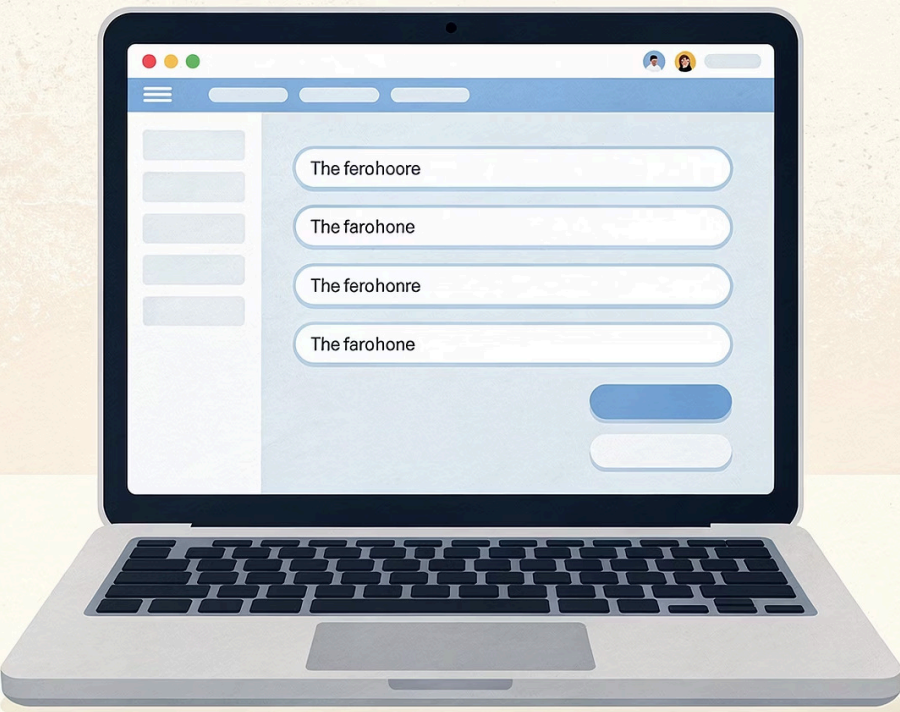
```
document.addEventListener('keydown', (e) => {  
  // Captura a tecla ESC  
  if (e.key === 'Escape') {  
    fecharModal();  
  }  
  // Captura Enter  
  if (e.key === 'Enter') {  
    confirmarAcao();  
  }  
  console.log('Tecla:', e.key);  
  console.log('Código:', e.code);  
});
```

keyup – Tecla Solta

Disparado quando o usuário **solta** a tecla. Não se repete. Preferível para validações e buscas, pois aguarda o término da digitação.

```
const input = document.querySelector('#busca');  
  
input.addEventListener('keyup', (e) => {  
  // Pesquisa ao soltar a tecla  
  const termo = e.target.value;  
  buscarResultados(termo);  
  
  // Propriedades úteis  
  console.log('Shift pressionado?', e.shiftKey);  
  console.log('Ctrl pressionado?', e.ctrlKey);  
});
```

❏ **Dica:** Use `e.key` para obter o caractere legível (ex: "Enter", "a") e `e.code` para a posição física da tecla no teclado.



CAPÍTULO 4

Eventos de Formulário

Formulários são o coração da coleta de dados na web. Os eventos de formulário permitem reagir a cada mudança do usuário em campos de texto, selects e checkboxes — desde a digitação em tempo real até o envio final.

input, change e submit – Qual usar?

input

Tempo real: dispara a cada caractere digitado. Use para validações imediatas, contadores de caracteres e buscas ao vivo.

```
campo.addEventListener('input', (e) => {  
  const val = e.target.value;  
  contador.textContent =  
    `${val.length}/100 chars`;  
  validarCampo(val);  
});
```

change

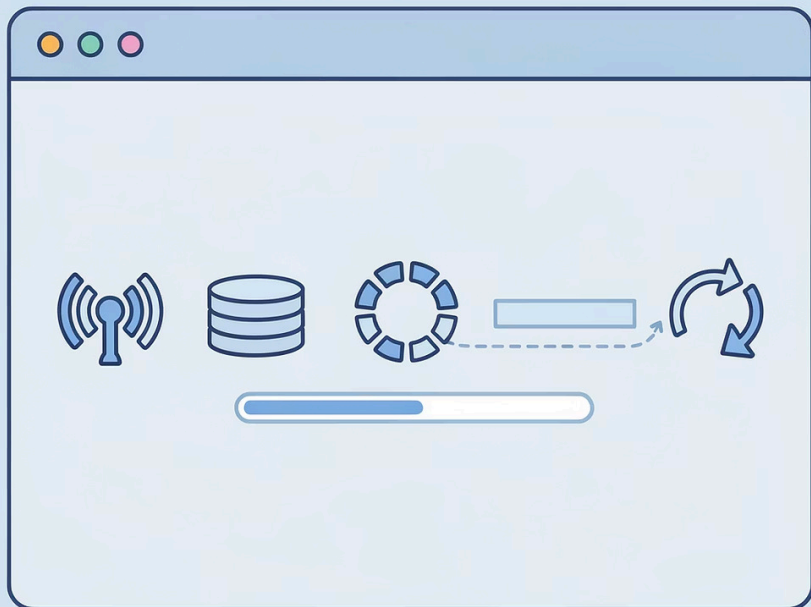
Após perda de foco: dispara quando o valor muda e o elemento perde o foco. Ideal para selects e checkboxes onde confirmar a seleção importa.

```
select.addEventListener('change', (e) =>  
{  
  const opcao = e.target.value;  
  filtrarTabela(opcao);  
});  
  
checkbox.addEventListener('change', (e)  
=> {  
  console.log('Marcado:',  
    e.target.checked);  
});
```

submit

Envio do formulário: dispara ao submeter. Sempre combine com `preventDefault()` para evitar o recarregamento da página e processar os dados via JS.

```
form.addEventListener('submit', (e) => {  
  //ESSENCIAL!  
  e.preventDefault();  
  
  const dados = new FormData(form);  
  enviarParaAPI(dados);  
});
```



CAPÍTULO 5

Eventos de Ciclo de Vida da Página

Além das interações do usuário, o JavaScript também reage a eventos do próprio navegador — como o carregamento da página, redimensionamento da janela e rolagem. Dominar esses eventos é crucial para performance e UX.

DOMContentLoaded, resize e scroll



DOMContentLoaded

Dispara quando o HTML foi completamente carregado e o DOM está pronto, **sem esperar** por imagens e estilos. O lugar certo para inicializar sua aplicação.

```
document.addEventListener(
  'DOMContentLoaded', () => {
    inicializarApp();
    console.log('DOM pronto!');
  }
);
```



resize

Dispara toda vez que a janela é redimensionada.

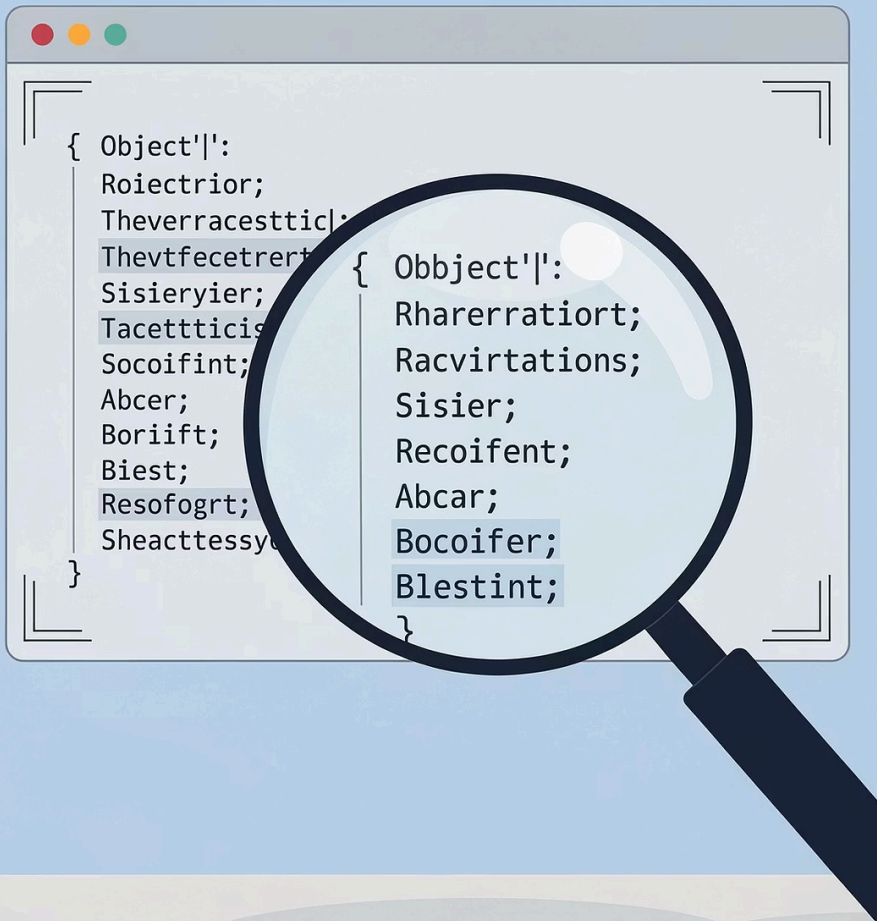
```
window.addEventListener('resize', () => {
  const largura = window.innerWidth;
  const altura = window.innerHeight;
  ajustarLayout(largura, altura);
});
```



scroll

Dispara durante a rolagem da página. Útil para navegações que aparecem ao rolar, carregamento lazy, barras de progresso e animações baseadas em scroll.

```
window.addEventListener('scroll', () => {
  const pos = window.scrollY;
  if (pos > 300) {
    btnTopo.classList.add('visivel');
  } else {
    btnTopo.classList.remove('visivel');
  }
});
```



CAPÍTULO 6

Deep Dive: O Objeto Event

Quando um evento é disparado, o navegador cria automaticamente um **Objeto Event** com todas as informações sobre o que aconteceu. Esse objeto é passado como parâmetro para o callback — convencionalmente chamado de `e` ou `event`.

O Parâmetro e ou event

O que ele contém?

O objeto Event é rico em informações. Algumas propriedades universais:

- `e.type` — o nome do evento ("click", "keydown"...)
- `e.target` — o elemento que originou o evento
- `e.currentTarget` — o elemento onde o listener está
- `e.timeStamp` — momento exato do evento

Explorando o Objeto Event

```
document.querySelector('#area')  
  .addEventListener('click', (e) => {  
  
  // Tipo do evento  
  console.log(e.type); // "click"  
  
  // Elemento clicado  
  console.log(e.target); //
```

```
// Elemento com o listener console.log(e.currentTarget); // Coordenadas do  
clique console.log(e.clientX, e.clientY); // Timestamp do evento  
console.log(e.timeStamp); // 1523.4 ms // Inspezione tudo: console.log(e); });
```

event.target – Identificando Elementos

A propriedade `event.target` retorna exatamente o elemento que **originou** o evento. Isso permite uma técnica poderosa chamada **Event Delegation**: adicionar um único listener a um elemento pai e tratar cliques em qualquer filho dinamicamente.

```
// SEM event delegation (ineficiente)
document.querySelectorAll('.item').forEach(item => {
  item.addEventListener('click', () => { /* ... */ });
});
```

```
// COM event delegation (eficiente e escalável)
const lista = document.querySelector('#lista');
```

```
lista.addEventListener('click', (e) => {
  // Verifica se clicou em um item
  if (e.target.matches('.item')) {
    console.log('Item clicado:', e.target.textContent);
    e.target.classList.toggle('selecionado');
  }
});
```

```
// Pega atributos de dados do elemento
const id = e.target.dataset.id;
console.log('ID do item:', id);
});
```

event.preventDefault()

Cancela o comportamento padrão do navegador para aquele evento. Indispensável em formulários, links e outros elementos com ações nativas.

```
// Evita reload no submit
form.addEventListener('submit', (e) => {
  e.preventDefault();
  processarFormulario();
});

// Evita navegação em link
link.addEventListener('click', (e) => {
  e.preventDefault();
  abrirModalPreview();
});

// Evita menu de contexto
area.addEventListener('contextmenu', (e) => {
  e.preventDefault();
  exibirMenuPersonalizado(e.clientX, e.clientY);
});
```


Recapitulando – O que você aprendeu



Eventos Fundamentais

`addEventListener` é a forma moderna de escutar eventos no DOM



Mouse e Teclado

`click`, `dblclick`, `mouseover`, `keydown`, `keyup` — interações essenciais



Formulários

`input` (tempo real), `change` (foco) e `submit` com `preventDefault()`



Ciclo de Vida

`DOMContentLoaded`, `resize` e `scroll` para controlar a página inteira



Objeto Event

`event.target` para controle total